Fine-Tuned Encoders or Prompted LLMs for Requirements Classification? The Verdict Inverts Under Distribution Shift

Fatma El-Zahraa Samir, Khaled T. Wassif, and Lamia AbouZeid
Faculty of Computers and Artificial Intelligence, *Cairo University*, Giza, Egypt
fatmaelzahraasamirabdelfattah@gmail.com

***Abstract*—Recent studies report that prompted large language models (LLMs) match or beat fine-tuned transformers on software requirements classification, and conclude that the task is close to solved. We show that this conclusion is an artefact of *how* the field evaluates. Twelve configurations—four fine-tuned encoder variants and eight prompted LLMs spanning 4-bit open-weight models on a free-tier GPU, a hosted 70 B model and a commercial API—classify 1,412 de-duplicated requirements from PROMISE_exp and SecReq across three tasks; the encoders are re-fitted at three regimes of increasing distribution shift—in-domain, cross-project, cross-dataset—while the prompted models, having nothing to fit, stay fixed. All 76,115 predictions are scored under one strict rule; every encoder–LLM comparison is McNemar-paired on identical items and corrected across a single 260-test family. In-domain the encoders win 67 of 106 tests and lose none. Under cross-dataset transfer the verdict inverts: prompted models win 48 of 64, and the best in-domain encoder loses 42.8% of its macro-F1. The cause is not unfamiliar vocabulary but an uncalibrated class prior—and both model families have it. The transferred encoder labels 44.2% of items *security* against a true prevalence of 12.9%, reproducing its training corpus's 39.9% prior; the prompted models' FR/NFR accuracy tracks how close their own prior is to the truth ($\rho = 0.95$), and re-wording the question moves it by up to 0.485. Encoders nevertheless stay 9–47$\times$ cheaper and repay their training within 46–552 items. Fine-tune for a stable labelled domain; prompt when the distribution moves.**

***Index Terms*—requirements classification, large language models, fine-tuning, generalisation, distribution shift, empirical software engineering, cost-efficiency**

I. Introduction

Requirements classification—assigning natural-language requirement statements to categories such as functional (FR) versus non-functional (NFR), or security-relevant versus not—is among the earliest analytical tasks in requirements engineering (RE). Getting it right supports early risk identification and architectural choice; getting it wrong propagates downstream as rework [1]. Two decades of automation have moved the task from weighted indicator terms [1], through supervised learning on engineered features [2], [4], to fine-tuned transformers [3] and, since 2024, to prompted generative LLMs [6], [10]–[12], [15].

The field's current headline is that the task is essentially solved. Fine-tuned encoders, commercial LLMs and even well-prompted small open models all report macro-F1 and accuracy in the high eighties to mid nineties [3], [10], [12], [14], [15]. Several studies conclude that prompting can replace supervision outright, removing the need for annotated data [10], [12].

That conclusion rests on a measurement convention. Almost every published result is *in-distribution*: the model is tuned, or its few-shot exemplars selected, on the same corpus it is then scored on. Where prior work claims improved "generalisability" it usually means transfer across *tasks* on the same two corpora, not across unseen projects or an independent dataset [10]—precisely the property the field's own foundational baseline warned was fragile, since NoRBERT was motivated by classifiers over-fitting project-specific wording [3]. A second omission compounds the first: inference cost, latency and model size are seldom quantified at all, though they decide whether a technique is adoptable without a commercial API budget.

This paper asks what happens to the published ranking when those two omissions are repaired. We hold the corpus, the items, the label sets and the scoring rule fixed, and vary only the amount of distribution shift between what a model was tuned on and what it is scored on. Concretely, we contribute:

1) **A paired, multiplicity-controlled benchmark** of twelve configurations—four fine-tuned encoder variants and eight prompted LLMs across local, hosted and commercial tiers—over three tasks, two corpora and three regimes: 76,115 predictions scored once under one strict protocol, with 260 exact McNemar tests corrected as a single family (Sections III and IV).
2) **The inversion result.** In-domain the encoders are never significantly beaten (67 wins, 39 ties, 0 losses). Under cross-dataset transfer—which the design can pose only on the security task, the one task both corpora annotate—the verdict reverses (3 wins, 13 ties, 48 losses), the strongest in-domain encoder there losing 42.8% of its macro-F1. The ranking a benchmark reports is a property of the regime it evaluates, not of the models (Section IV-B).
3) **A mechanism, not just a gap.** The collapse is consistent with base-rate shift rather than lexical novelty—the transferred encoder reproduces the prior of its *training* corpus, so precision collapses while recall holds—and the prompted models prove to have the same defect from another source: their FR/NFR accuracy tracks how close their own prior is to the truth ($\rho = 0.95$), and re-wording the question moves it (Sections IV-B and IV-C).

4) **An explicit accuracy–cost frontier.** Encoders cost \$0.0024 per 1,000 classifications against \$0.0216–\$0.1132 for prompted alternatives, and repay their one-off fine-tuning within 46–552 classifications in all 276 measured pairings—which is what makes the transfer finding a real trade-off rather than a verdict (Section IV-D).

Three research questions organise the study. **RQ1**: how do open and commercial LLMs compare with fine-tuned encoders in-domain? **RQ2**: how much performance is lost under cross-project and cross-dataset evaluation, and does the loss differ by model class? **RQ3**: what is the accuracy-versus-cost trade-off, and what is the cheapest freely available option that is good enough?

II. Related Work and the Evaluation Gap

A. From indicator terms to prompting

Automated requirements classification has passed through four overlapping waves. Rule-based work established the task and produced the PROMISE NFR collection [1]; classical supervised learning followed, with SVMs separating FRs from NFRs [2] and interpretable variants combining dependency parsing with feature attribution [4]; deep models removed much of the feature engineering but demanded more data [8]. Transfer learning then reset the state of the art: NoRBERT fine-tunes BERT and reaches macro-F1 in the mid nineties [3], a pattern a systematic mapping later confirmed [7]. Zero-shot sentence-embedding methods showed labelled data was not strictly necessary [5], and prompted generative LLMs now dominate.

B. Recent LLM studies (2024–2026)

El-Hajjami et al. [6] compared SVM and LSTM against GPT-3.5 and GPT-4 over five datasets and found no single best technique; Peer et al. [9] embedded LLM classification in a model-based systems engineering workflow. Binkhonain and Alfayaz [10] report the study closest to ours: five LLMs under four prompting strategies over three tasks on PROMISE and SecReq, benchmarked against a fine-tuned transformer that few-shot prompting matches or exceeds. Alhoshan et al. [11] ran more than 400 experiments over three open generative models, and Tang et al. [12] identified an "over-prompting" effect in which extra in-context examples degrade accuracy on PROMISE and PURE [18], letting a TF–IDF exemplar selector push an 8 B open model past the fine-tuned baseline. Chaudhary et al. [13] extended the task to noisy app-store reviews, Vasudevan et al. [14] compared fine-tuned BERT and RoBERTa against GPT-4o, and Levy et al. [15] benchmarked three current models against expert judgment on PROMISE_exp.

C. What the field does not measure

Table I summarises the evaluation coverage of these studies along the axes that matter for adoption. Three patterns are near-universal. *First*, evaluation is in-distribution: the same corpus supplies the training data or the few-shot exemplars and the test items. Only the app-review study [13] tests genuinely different text, and it records markedly lower precision on several categories—an unexplained signal that the field has not followed up. *Second*, cost is not a reported dimension; work that does use open models focuses on predictive performance alone [11], [12]. *Third*, the same two or three small, ageing, public corpora underlie nearly every result, so contamination of LLM pre-training data is a live threat that is acknowledged but never addressed [33].

TABLE I
Evaluation coverage of recent LLM-based requirements-classification studies. ✓ = reported; ✗ = not reported. "Cross-project" means held-out unseen projects; "cross-dataset" means an independently produced corpus.

Study	Open LLMs	Comm. LLMs	Cross-project	Cross-dataset	Cost
El-Hajjami et al. [6]	✗	✓	✗	✗	✗
Peer et al. [9]	✗	✓	✗	✗	∼
Binkhonain & Alfayaz [10]	✓	✓	✗	✗	✗
Alhoshan et al. [11]	✓	✗	✗	✗	✗
Tang et al. [12]	✓	✓	✗	✗	✗
Chaudhary et al. [13]	✗	✓	✗	∼	✗
Vasudevan et al. [14]	✗	✓	✗	✗	✗
Levy et al. [15]	✓	✓	✗	✗	✗
This paper	✓	✓	✓	✓	✓

These gaps are not independent. If in-distribution parity is what motivates replacing supervision with prompting, and is also exactly what contamination and corpus reuse would produce spuriously, then the comparison that settles the question has not yet been run. This paper runs it.

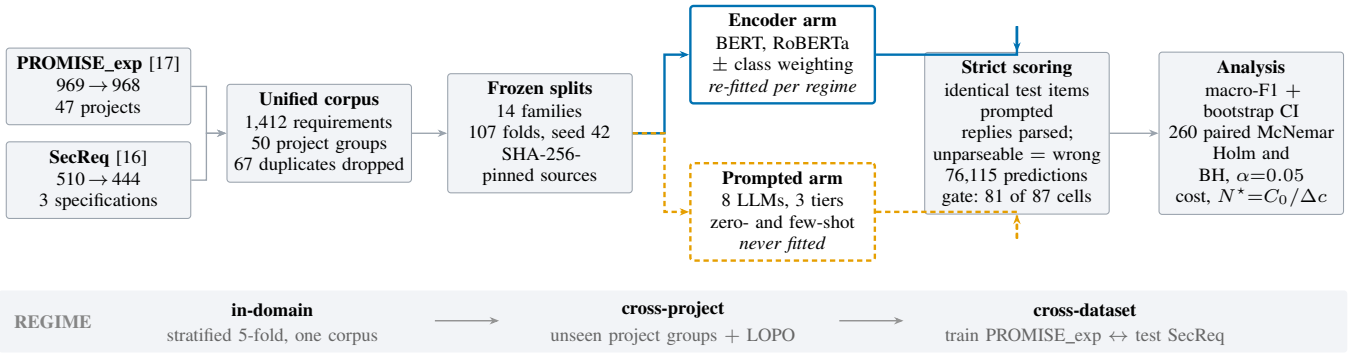
III. Study Design

The study is a controlled benchmark. Twelve model configurations classify the same requirements, under the same tasks, against the same frozen splits, and are scored by one strict rule, so that every comparison we report is paired on identical items. Fig. 1 shows the design: a single data and scoring spine, two model arms that differ only in whether they consume training data, and three evaluation regimes of increasing distribution shift. The pipeline runs as seven stages, each writing a versioned artefact that the next stage consumes; every number in Section IV is read back from those artefacts.

A. Corpus

Two public corpora are merged (Table II). PROMISE_exp [17] contributes 969 requirements from 47 projects, SecReq [16] 510 from three industrial security specifications. Texts are normalised and exact duplicates removed—67 rows, none shared between corpora—leaving 1,412 requirements in 50 project groups. Splits are generated once with seed 42 and keyed by stable item identifiers over source files that the manifest pins by SHA-256; because grouped-fold membership depends on the library version that produced it [22], the split file is consumed, never regenerated.

The decisive property of this pair is a base-rate contrast: security requirements are 12.9% of PROMISE_exp but 39.9%



Solid (upper) path: consumes training folds, re-fitted at every station of the regime axis. **Dashed (lower) path:** consumes only test items, so it is regime-invariant and forms the fixed reference against which the encoders' drop is measured.

Fig. 1. Experimental architecture. One data spine (left) feeds two model arms that differ in exactly one respect: whether they consume training data. Both answer the *same* items—so every encoder–LLM comparison is paired item-for-item—and both pass through one strict scoring spine (right). The band beneath is the study's independent variable: the distribution shift between what a model was fitted on and what it is scored on.

TABLE II
CORPUS CONSTRUCTION. PREVALENCE IS THE SHARE OF ITEMS IN THE POSITIVE CLASS OF THE SECURITY TASK; THE THREE-FOLD CONTRAST BETWEEN THE TWO CORPORA IS WHAT THE CROSS-DATASET ARM EXPLOITS.

Corpus	Raw	Final	Groups	FR/NFR	Sec. prev.
PROMISE_exp [17]	969	968	47	444 / 524	12.9%
SecReq [16]	510	444	3	—	39.9%
Unified	1,479	1,412	50	444 / 524	21.4%

of SecReq—a three-fold difference in prevalence for a concept both corpora annotate independently. That contrast, not vocabulary, drives the main result.

B. Tasks

Three task families are defined over the unified corpus. *Binary FR/NFR* uses the native PROMISE_exp labels. *Binary security* is native in SecReq and derived in PROMISE_exp from the SE sub-class, giving the same task on two independently produced corpora—the property the cross-dataset arm exploits. *NFR sub-type* classification uses the eleven-class taxonomy of Cleland-Huang et al. [1], which aligns with the ISO/IEC 25010 quality characteristics [23]; because several tail classes have single-digit support, reduced top-6 and top-4 variants are also evaluated (524, 433 and 353 items). macro-F1 is always computed over the *full declared label set* of the variant, so a model is penalised for every class it never predicts.

C. Evaluation regimes

Fourteen frozen split families (107 folds) implement three regimes of increasing shift. **In-domain** is stratified 5-fold cross-validation within one corpus—the in-distribution reference arm prior work reports. **Cross-project** uses grouped folds holding out entire unseen project groups, plus a leave-one-project-out arm over the 47 PROMISE_exp projects. **Cross-dataset** trains on the security task of one corpus and tests on the other (PROMISE_exp ↔ SecReq): identical label sets, different annotation provenance.

The asymmetry between the two model arms is the analytical lever. Encoders are re-trained at every station on that station's training folds. Prompted models have no training distribution, so their scores are regime-invariant by construction: they form a fixed reference line against which the encoders' drop is measured. Every requirement is a test item exactly once within each encoder family, so the encoders' stored predictions cover the corpus; the prompted models answer stratified samples of that same corpus—a 300-item core per binary cell and a 200-item core per sub-type cell (218 for the eleven-class variant), allocated in proportion to the class prior with a floor of ten per class. The six local models complete those cores and additionally answer the full evaluation frames; the rate-limited hosted and commercial arms complete as much of each core as their quotas allowed, so prompted cell sizes range from 59 to 960. Every encoder–LLM comparison is paired on identical requirements, aligned by requirement identifier rather than by position.

D. Model configurations

Table III lists the twelve configurations. The encoder tier fine-tunes `bert-base-uncased` [19] and `roberta-base` [20] with a classification head (AdamW, lr 2×10^{-5}, batch 16, max length 128, linear decay without warmup, full precision; 3 epochs on the binary tasks, 10 on the sub-types), each with inverse-frequency (*balanced*) class weighting in the loss, plus an unweighted control on the security task's in-domain PROMISE_exp and cross-dataset families. The local open tier runs six instruction-tuned models of 2.6–8.0 B parameters on a single free-tier NVIDIA T4 under 4-bit NF4 quantisation [21]. The hosted tier accesses Llama-3.3-70B and the commercial tier Gemini 3.1 Flash-Lite, both through free API tiers. A ninth prompted model was attempted but returned only 21 items per cell and failed the reportability gate below; its predictions stay in the raw store and enter no table.

E. Prompting and strict parsing

Each prompted model answers a fixed instruction naming the task, enumerating the label vocabulary and demanding

TABLE III
THE TWELVE MODEL CONFIGURATIONS. GEMINI'S PARAMETER COUNT IS UNDISCLOSED AND IS RECORDED AS UNKNOWN RATHER THAN ESTIMATED.

Config.	Checkpoint / endpoint	B	Access
BERT (w / u)	`bert-base-uncased`	0.11	fine-tuned
RoBERTa (w / u)	`roberta-base`	0.13	fine-tuned
Gemma-2-2B	`gemma-2-2b-it`	2.61	local, 4-bit
SmolLM3-3B	`SmolLM3-3B`	3.08	local, 4-bit
Qwen2.5-3B	`Qwen2.5-3B-Instruct`	3.09	local, 4-bit
Phi-4-mini	`Phi-4-mini-instruct`	3.84	local, 4-bit
Qwen2.5-7B	`Qwen2.5-7B-Instruct`	7.62	local, 4-bit
Llama-3.1-8B	`Llama-3.1-8B-Instruct`	8.03	local, 4-bit
Llama-3.3-70B	`llama-3.3-70b-versatile`	70.6	hosted
Gemini 3.1 F-Lite	`gemini-3.1-flash-lite`	n/a	commercial

```
You are classifying a software requirement.
Decide whether it is a FUNCTIONAL requirement (FR) --
something the system must DO -- or a NON-FUNCTIONAL
requirement (NFR) -- a quality, constraint or property
such as performance, security or usability.
Answer with exactly one word: "FR" or "NFR".
⟨few-shot conditions only⟩
Examples:
Requirement: """exemplar"""                    Answer: FR
... k class-balanced exemplars, carved out before the evaluation frames
Now classify this one:
Requirement:
"""the requirement under test"""
Answer:
```

Fig. 2. The *base* FR/NFR instruction, verbatim, and the position at which few-shot exemplars are injected. The *terse* and *verbose* phrasings vary only the instruction block: the answer contract—exactly one word—is identical across all three, and is what the strict parser enforces.

exactly one word in reply, sent as a single user message (Fig. 2). Decoding is greedy—temperature 0, at most twelve new tokens—so the only stochastic element in this arm is item sampling. Few-shot conditions insert deterministic, class-balanced exemplars ($k \in \{2, 4, 8\}$ in total on the binary tasks, $k \in \{1, 2\}$ per class on the sub-types) carved out of the corpus *before* the evaluation frames are formed, so an exemplar is never also a test item. Two sub-studies vary one factor at a time—the phrasing (*base*, *terse*, *verbose*) and the few-shot ladder; the hosted and commercial arms run zero-shot only, under provider rate limits.

Outputs are parsed by a strict, word-boundary-aware rule: a leading label wins; otherwise exactly one label named anywhere in the reply wins; a reply naming both labels or neither is *unparseable and scored as wrong*. This is deliberately harsher than the lenient protocols common in prior work, and it matches deployment, where an unusable answer is not a free pass. The same rule is applied at generation time and again in an idempotent repair pass over the archived raw outputs, so stored and scored predictions cannot diverge silently.

F. Metrics, reportability, and significance

The primary metric is macro-F1 over the declared label set, with weighted F1 and accuracy alongside. Uncertainty uses a stratified bootstrap [24]: 2,000 resamples drawn within each true class (seed 42), so no resample can lose a rare class. Encoder–LLM comparisons use the exact two-sided McNemar test on paired disagreements [25], [26]; all 260 are corrected as one family under both Holm [27] and Benjamini–Hochberg [28] at $\alpha = 0.05$, with verdicts keyed on the BH-adjusted p (175 significant raw, 170 after BH, 131 after Holm). A cell (model $\times$ task $\times$ dataset $\times$ regime) is reported only if every class has ≥ 5 test items, the cell holds ≥ 40 items, and the model parsed $\geq 50\%$ of its responses; 6 of 87 cells fail this gate and are excluded rather than imputed. Majority-class and stratified-random baselines run through the same scorer. The store holds 76,115 predictions: 24,280 encoder, 35,049 prompted binary, 16,786 prompted sub-type.

G. Cost model

Every prediction row records tokens, wall-clock latency and, for encoders, training time. Local computation is priced as an imputed rental of the benchmark GPU (AWS `g4dn.xlarge`, one NVIDIA T4, \$0.5260/h on demand); API models at provider list token rates (Gemini 3.1 Flash-Lite \$0.25/\$1.50 per million input/output tokens, Llama-3.3-70B \$0.59/\$0.79), frozen into each row at generation time. Encoder cost separates the one-off fine-tuning cost C_0 from the marginal per-item cost c_{enc}, giving a break-even volume against any prompted alternative

$$N^{\star} = \frac{C_0}{c_{\text{alt}} - c_{\text{enc}}}. \tag{1}$$

Because free-tier latency is dominated by provider-side queueing, wall-clock projections use per-request median latency, with the queue-inflated mean reported separately.

IV. RESULTS

A. RQ1: in-domain, supervision still wins

Table IV reports in-domain macro-F1 for all twelve configurations over the six task $\times$ dataset cells. Fine-tuned encoders post the best score in five of six columns: class-weighted BERT reaches 0.908 on FR/NFR and 0.923 on PROMISE_exp security, and class-weighted RoBERTa reaches 0.936 / 0.837 / 0.724 on the top-4 / top-6 / all-11 sub-type variants. The single exception is SecReq security, where 4-bit Qwen2.5-7B posts the better point estimate (0.846 against 0.836)—a lead that never reaches significance (exact McNemar $p = 0.696$; BH-adjusted 0.774).

Three observations qualify the ranking. Scale does not order the results: the 2.6 B Gemma-2-2B beats the hosted 70 B model on PROMISE_exp security (0.900 against 0.891), and the small open models collapse on FR/NFR (0.554–0.727) while remaining competitive on security. The encoder advantage widens with the label set—$+0.014$ on top-4, $+0.013$ on top-6, but $+0.095$ on all-11—as expected if supervision mainly buys the long tail. And because cell sizes differ ($n = 59$–968), the top-4 ranking was recomputed on the 59 items every model answered, where the best encoder still leads the best prompted model by $+0.036$ (0.966 against 0.930): the ordering is not an artefact of coverage. Every reportable model clears the majority-class baseline.

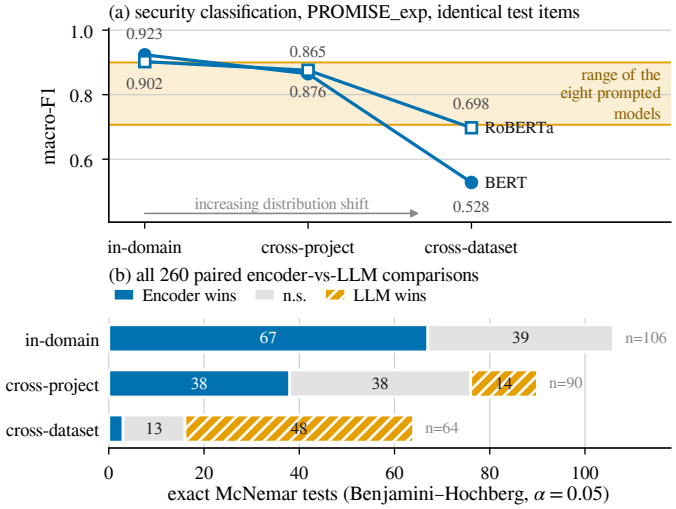


Fig. 3. The inversion. (a) On the security task over identical PROMISE_exp items, both encoders start well above the range spanned by the eight prompted models and fall through it as distribution shift increases. (b) The same reversal in the paired tests: 260 exact McNemar comparisons, split by the regime the encoder was evaluated under.

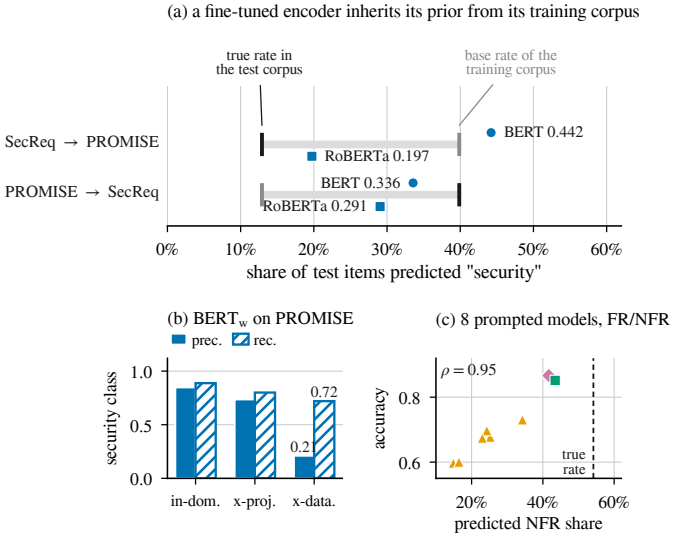


Fig. 4. One failure mode, three sources. (a) After cross-dataset transfer an encoder labels items at a rate pulled toward the base rate of the corpus it was *trained* on. (b) The consequence: security-class precision collapses while recall is largely preserved—the signature of prior shift, not of unfamiliar vocabulary. (c) The prompted models have the same defect from another source: on FR/NFR every one of them under-predicts the NFR class, and the closer a model's predicted share is to the true rate the better it scores.

The paired tests make the in-domain verdict exact: over 106 in-domain comparisons, encoders win 67 and lose *none* (Fig. 3 (b)).

B. RQ2: the verdict inverts under distribution shift

Table V lists every measured transfer, and Fig. 3 shows the result in both metric and verdict form.

Cross-project transfer is survivable. Holding out unseen projects costs the encoders 3–12% of their macro-F1 on PROMISE_exp and up to 30% on the much smaller SecReq, but leaves them ahead or tied in 76 of 90 paired tests—the degradation NoRBERT anticipated [3], real but moderate.

Cross-dataset transfer reverses the ranking. The class-weighted BERT that scores 0.923 on PROMISE_exp security falls to 0.528 on the *very same items* once its training corpus is swapped for SecReq—a loss of 42.8%. RoBERTa is more robust (0.902 → 0.698, −23%) and the reverse direction milder still (−17%), while the prompted models, tuning on nothing, move only with corpus difficulty (−0.034 to +0.163). Prompted models win 48 of the 64 cross-dataset comparisons and encoders 3: in-domain 67–39–0 becomes 3–13–48 (Fig. 3 (b)), the same twelve models and the same scorer, the opposite conclusion.

Aggregate scores hide per-project failure. On the leave-one-project-out arm of FR/NFR, across the 37 projects with comparable coverage, *every* configuration—encoder and prompted alike—has at least one project it scores 0.000 on and one it scores 1.000 on; only the middle separates them (encoder median 0.917, s.d. 0.198, against 0.545–0.750 and s.d. 0.213–0.301 for the small open models). A corpus-level average is a weak predictor of what a practitioner meets on their own project.

The mechanism is base-rate shift. A performance drop alone does not say *why* a model fails, and the two candidate explanations—unfamiliar vocabulary and shifted class priors—imply different remedies. The per-class evidence separates them (Fig. 4). The transferred BERT labels 44.2% of PROMISE_exp items *security* against a true prevalence of 12.9%, close to the 39.9% prior of the corpus it was trained on. Its security-class precision collapses from 0.847 to 0.210 while recall falls far less (0.888 → 0.720): the model still finds the security requirements, it simply cannot stop finding them. That is the signature of prior shift, not of a model that has failed to understand the text.

The picture is not uniform, and we report it as measured. RoBERTa carries less of its source prior across (19.7% predicted against 12.9% true) and loses correspondingly less accuracy, and in the reverse direction both encoders predict *below* the target prevalence (33.6% and 29.1% against 39.9%), so content differences contribute there too. What the evidence supports is that the dominant term in the largest observed collapse is a prior the model brought with it—an encouraging diagnosis, because prior shift is correctable: threshold or prior adjustment [29], [30] needs no labels from the target corpus. Applying it would change the quantity being measured, so we leave it as the obvious next step.

C. One failure mode, three sources

Section IV-B attributed the encoders' collapse to a prior carried over from training. The same instrument, turned on the prompted arm, finds the same defect there—arriving by a different route.

Prompted models also mis-set the prior, and it explains most of their error. On FR/NFR the true NFR share is 54.2%, yet every prompted model under-predicts it—from 14.9%

TABLE IV

RQ1—IN-DOMAIN MACRO-F1 FOR ALL TWELVE CONFIGURATIONS. BEST PER COLUMN IN BOLD; "—" MARKS A CELL BELOW THE REPORTABILITY GATE OR NOT RUN. ENCODER CELLS ARE 5-FOLD CROSS-VALIDATED OVER ALL ITEMS (n = 968, 968, 444, 353, 433, 524); PROMPTED CELLS ARE STRATIFIED SAMPLES OF THOSE SAME ITEMS, SO EVERY COMPARISON IN FIG. 3 IS PAIRED ITEM-FOR-ITEM.

	FR/NFR	**Security**		**NFR sub-types (PROMISE_exp)**		
Configuration	PROMISE_exp	PROMISE_exp	SecReq	top-4	top-6	all-11
BERT_{w}	**0.908**	**0.923**	0.836	0.878	0.827	0.657
BERT_{u}	–	0.907	–	–	–	–
$\text{RoBERTa}_{\text{w}}$	0.896	0.902	0.824	**0.936**	**0.837**	**0.724**
$\text{RoBERTa}_{\text{u}}$	–	0.903	–	–	–	–
Gemini 3.1 Flash-Lite	0.867	0.829	0.713	0.922	–	–
Llama-3.3-70B	0.852	0.891	0.814	0.907	0.782	–
Qwen2.5-7B	0.656	0.884	**0.846**	0.876	0.824	0.576
Llama-3.1-8B	0.682	0.872	0.845	0.700	0.717	0.629
Gemma-2-2B	0.663	0.900	0.821	0.760	0.603	0.527
Phi-4-mini	0.727	0.798	0.635	0.727	0.712	0.599
SmolLM3-3B	0.562	0.841	0.717	0.763	0.693	0.482
Qwen2.5-3B	0.554	0.707	0.741	0.732	0.689	0.543
majority-class baseline	0.351	0.466	0.376	0.131	0.075	0.035

TABLE V

RQ2—ENCODER MACRO-F1 BY EVALUATION REGIME, WITH THE RELATIVE LOSS AGAINST THE IN-DOMAIN REFERENCE. CROSS-DATASET APPLIES ONLY TO THE SECURITY TASK, THE ONE TASK BOTH CORPORA ANNOTATE.

	In-dom.	**Cross-project**		**Cross-dataset**	
Configuration	macro-F1	macro-F1	$\Delta\%$	macro-F1	$\Delta\%$
Security — PROMISE_exp					
BERT_{w}	0.923	0.865	-6	0.528	-43
BERT_{u}	0.907	–	–	0.534	-41
$\text{RoBERTa}_{\text{w}}$	0.902	0.876	-3	0.698	-23
$\text{RoBERTa}_{\text{u}}$	0.903	–	–	0.707	-22
Security — SecReq					
BERT_{w}	0.836	0.588	-30	0.690	-17
$\text{RoBERTa}_{\text{w}}$	0.824	0.668	-19	0.696	-16
FR/NFR — PROMISE_exp					
BERT_{w}	0.908	0.843	-7	–	–
$\text{RoBERTa}_{\text{w}}$	0.896	0.853	-5	–	–
NFR sub-types, top-4					
BERT_{w}	0.878	0.800	-9	–	–
$\text{RoBERTa}_{\text{w}}$	0.936	0.835	-11	–	–
NFR sub-types, top-6					
BERT_{w}	0.827	0.735	-11	–	–
$\text{RoBERTa}_{\text{w}}$	0.837	0.753	-10	–	–
NFR sub-types, all-11					
BERT_{w}	0.657	0.579	-12	–	–
$\text{RoBERTa}_{\text{w}}$	0.724	0.688	-5	–	–

(Qwen2.5-3B) to 43.4% (Llama-3.3-70B)—and accuracy tracks that shortfall almost exactly: predicted share and accuracy rank together at Spearman $\rho = 0.95$ over the eight reportable models (Fig. 4 (c)). The small open models' apparent "collapse" on FR/NFR is thus not an inability to read the requirement but a systematic bias toward answering FR.

The prompt wording moves the prior directly—which makes the link causal rather than correlational. Across the nine paired model $\times$ task cells of the phrasing sub-study the median spread between the *base*, *terse* and *verbose* wordings is only 0.044 macro-F1, but the extreme is 0.485, for Gemma-2-2B on SecReq security (0.827 / 0.343 / 0.828). The predictions show what the terse wording does: it is the only variant phrased as a leading yes/no question, and under it Gemma-2-2B answers *security* for 96.7% of SecReq and 51.3% of PROMISE_exp items against true rates of 39.7% and 12.7%; Qwen2.5-7B shifts the same way but less far. This is not a formatting artefact—the strict parser accepted 99.98% of the sub-study's 11,518 responses—so the answers were well-formed and simply wrong. Rewording the question re-set the model's prior exactly as re-training on another corpus re-set the encoder's, the instability behind calibration and ordering fixes in the prompting literature [31], [32].

Few-shot exemplars help precisely where they carry prior information. Across the 90 paired comparisons on the six local models—the only tier that ran few-shot—the median gain is $+0.024$ macro-F1, but the effect splits by task: it is net-negative on the binary tasks (mean -0.007 over 54 comparisons; the 30 BH-significant effects average -0.027) and uniformly positive on the sub-types (mean $+0.062$ over 36; all 14 BH-significant effects are gains, averaging $+0.114$). Where the labels are a familiar convention the model already has a prior and examples mostly perturb it—Tang et al.'s over-prompting effect [12]; where they are an unfamiliar eleven-class taxonomy, the exemplars are the only statement of what the classes are.

Read together, these are one defect with three sources. This task needs a class prior that cannot be read off the requirement text, and each approach acquires one somewhere it should not: the encoder from its training corpus, the prompted model from its pre-training and from the wording of the question. Neither family is calibrated to the corpus in front of it—which is why neither is safe to deploy on an in-distribution score alone.

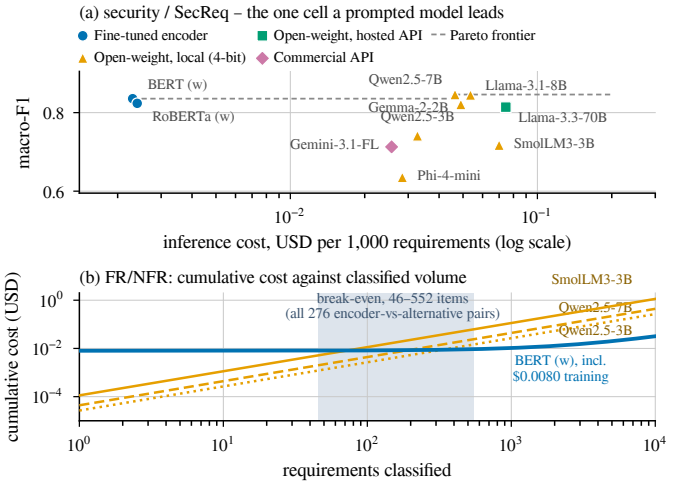


Fig. 5. RQ3. (a) SecReq security, the one cell a prompted model leads: 4-bit Qwen2.5-7B buys $+0.011$ macro-F1 for $\approx 19\times$ the cost per item. (b) Training is repaid within 46–552 classifications across all 276 pairings.

D. RQ3: the accuracy–cost frontier

Fig. 5 gives the economics. Encoder inference costs \$0.0024 per 1,000 classifications on the imputed T4 rental against \$0.0216–\$0.1132 for prompted alternatives—9 to 47 times more, 18 times at the medians. One-off fine-tuning is negligible (median \$0.0093 per deployable model, about 277 GPU-seconds over all folds of a family), so the break-even of Eq. (1) is small: over all 276 encoder-versus-alternative pairings $N^\star$ ranges from 46 to 552 items (median 215). The Pareto frontier over reportable cells holds ten cells, nine of them encoders; the single prompted frontier point is 4-bit Qwen2.5-7B on SecReq security—the strongest freely available fallback on the one cell where transfer is what is measured.

Latency behaves differently. Median per-request latency is 0.016 s for the encoders and, across the prompted models, 0.095 s for the hosted 70 B model (the fastest prompted tier), 0.19–0.36 s for the local open models and 0.61 s for the commercial API; but free-tier queueing inflates the hosted *means* by up to two orders of magnitude, which is why all wall-clock projections use medians.

V. Discussion

The regime is part of the claim. The most consequential finding here is not that one model class is better; it is that "which is better?" flips completely between two protocols the literature treats as interchangeable, on the same corpora, the same items and the same scorer. A benchmark reporting only in-distribution numbers is not a weaker version of one that reports transfer—it can be one that reports the opposite conclusion. Requirements-classification results should therefore state the regime as part of the claim, and "generalisation" should mean held-out projects or independent corpora, not transfer across tasks on one dataset.

A decision rule for practitioners. With a few hundred labelled requirements from the domain a team will operate in, and any non-trivial volume, a fine-tuned 110 M-parameter encoder is both the most accurate and by far the cheapest option. Entering an unlabelled or shifting domain, a mid-size quantised open model is safer: it gives up in-domain accuracy, degrades gracefully, and runs free on one consumer-class GPU. The worst option is the implicit default of much recent work—fine-tuning on one corpus and assuming the result transfers.

Contamination cuts in our favour. Both corpora are old, small and public, so prompted in-domain scores may be inflated by pre-training contamination [33]. That bias *overstates* the prompted models in-domain—exactly where we report the encoders winning—so the encoder advantage is conservative; and it cannot produce the cross-dataset reversal, since it would help them in both regimes equally.

Availability. The pipeline, the frozen splits, the full prediction store and the scripts that regenerate every table and figure from it are public; because scoring is a pure function of the stored raw outputs, every number here reproduces without re-running a model.

VI. Threats to Validity

Internal. The commercial arm ran at $n \approx 60$ per cell, where 23 of its 26 paired comparisons are inconclusive, so "never significantly beaten" partly reflects limited power. Free-tier quotas caused missing-not-at-random dropout on some hosted configurations; the gate excludes the affected cells rather than imputing them. Local models ran 4-bit quantised, which may understate full-precision accuracy, and each configuration used a single seed, so small encoder-to-encoder margins should not be over-read.

Construct. GPU costs are imputed from a published rental rate, not billed, and token prices are 2026 list prices—the ratios are more durable than the absolutes. $N^\star$ is a pure cost crossover computed regardless of the accuracy gap, and the model prices computation only: the human annotation cost an encoder needs is excluded, which favours the encoder. Strict scoring counts format non-compliance as error, matching deployment but differing from lenient protocols.

External. Both corpora are small, dated and public, and their security label sets differ—PROMISE_exp's derives from the SE sub-class and is non-functional by construction, whereas SecReq annotates any security-relevant sentence—so the cross-dataset arm carries definitional as well as distributional shift, on one corpus pair. Results cover English requirements and one NFR taxonomy.

Conclusion. All paired tests are exact and corrected as a single family.

VII. Conclusion

In-domain the fine-tuned encoders are never significantly beaten; across corpora the prompted models win 48 of the 64 paired tests; and encoders stay an order of magnitude cheaper. Both families carry an uncalibrated class prior—the encoder from its training corpus, the prompted model from the wording of the question. Two protocols the field treats as interchangeable therefore return opposite rankings from the same data: an in-distribution score is not a partial answer—it can be the wrong one.

REFERENCES

[1] J. Cleland-Huang, R. Settimi, X. Zou, and P. Solc, "Automated classification of non-functional requirements," *Requirements Eng.*, vol. 12, no. 2, pp. 103–120, 2007.

[2] Z. Kurtanović and W. Maalej, "Automatically classifying functional and non-functional requirements using supervised machine learning," in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 490–495.

[3] T. Hey, J. Keim, A. Koziolek, and W. F. Tichy, "NoRBERT: Transfer learning for requirements classification," in *Proc. IEEE 28th Int. Requirements Eng. Conf. (RE)*, 2020, pp. 169–179.

[4] F. Dalpiaz, D. Dell'Anna, F. B. Aydemir, and S. Çevikol, "Requirements classification with interpretable machine learning and dependency parsing," in *Proc. IEEE 27th Int. Requirements Eng. Conf. (RE)*, 2019, pp. 142–152.

[5] W. Alhoshan, A. Ferrari, and L. Zhao, "Zero-shot learning for requirements classification: An exploratory study," *Inf. Softw. Technol.*, vol. 159, art. 107202, 2023.

[6] A. El-Hajjami, N. Fafin, and C. Salinesi, "Which AI technique is better to classify requirements? An experiment with SVM, LSTM, and ChatGPT," arXiv:2311.11547, 2024.

[7] K. Kaur and P. Kaur, "The application of AI techniques in requirements classification: A systematic mapping," *Artif. Intell. Rev.*, vol. 57, art. 57, 2024.

[8] S. N. Sonawane and S. M. Puthran, "Classification of functional and nonfunctional requirements based on convolutional neural network with flower pollination optimizer," *Innov. Syst. Softw. Eng.*, 2024.

[9] J. Peer, Y. Mordecai, and Y. Reich, "NLP4ReF: Requirements classification and forecasting—From model-based design to large language models," in *Proc. IEEE Aerospace Conf.*, 2024, pp. 1–16.

[10] M. Binkhonain and R. Alfayaz, "Are prompts all you need? Evaluating prompt-based large language models for software requirements classification," *Requirements Eng.*, 2025.

[11] W. Alhoshan, A. Ferrari, and L. Zhao, "How effective are generative large language models in performing requirements classification?" arXiv:2504.16768, 2025.

[12] Y. Tang, D. Tuncel, C. Koerner, and T. Runkler, "The few-shot dilemma: Over-prompting large language models," arXiv:2509.13196, 2025.

[13] M. Chaudhary, C. Jain, and P. R. Anish, "Exploring zero-shot app review classification with ChatGPT: Challenges and potential," arXiv:2505.04759, 2025.

[14] P. Vasudevan, S. Reddivari, S. Ahuja, N. Niu, S. Kumar, B. Jung, and E. Gamess, "Requirements classification using fine-tuned transformer models: A comparative study of BERT, RoBERTa, and GPT-4o," in *Proc. ACM Southeast Conf. (ACMSE)*, 2026, pp. 199–204.

[15] O. Levy, I. Dikman, N. Levy, and M. Winokur, "AI-assisted requirements engineering: An empirical evaluation relative to expert judgment," arXiv:2604.15222, 2026.

[16] E. Knauss, S. Houmb, K. Schneider, S. Islam, and J. Jürjens, "Supporting requirements engineers in recognising security issues," in *Proc. REFSQ*, 2011, pp. 4–18.

[17] M. Lima, V. Valle, E. Costa, F. Lira, and B. Gadelha, "Software engineering repositories: Expanding the PROMISE database," in *Proc. XXXIII Brazilian Symp. Softw. Eng. (SBES)*, 2019, pp. 427–436.

[18] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A dataset of public requirements documents," in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 502–505.

[19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[20] Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[21] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 10088–10115.

[22] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[23] *ISO/IEC 25010:2011, Systems and Software Engineering—SQuaRE—System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.

[24] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1993.

[25] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[26] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.

[27] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[28] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. R. Statist. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.

[29] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, Eds., *Dataset Shift in Machine Learning*. Cambridge, MA, USA: MIT Press, 2009.

[30] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Comput.*, vol. 14, no. 1, pp. 21–41, 2002.

[31] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proc. ICML*, 2021, pp. 12697–12706.

[32] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," in *Proc. ACL*, 2022, pp. 8086–8098.

[33] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. L. de Lacalle, and E. Agirre, "NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark," in *Findings of EMNLP*, 2023, pp. 10776–10787.